UTEQ / ISR-701 / FORM-FB-01

Melanie Michelle Muñoz Vera · Semana 2, 2026

Ficha 1. Heterogeneidad tecnológica

Referencia IEEE

[1] G.-D. Schwarz, P. Heltweg y D. Riehle, «Balancing technology heterogeneity in microservice architectures,» *Empirical Software Engineering*, vol. 30, n.º 5, 2025, art. 127, doi: 10.1007/s10664-025-10684-4

Identificador verificado

DOI 10.1007/s10664-025-10684-4

Dato concreto y ubicación

El análisis temático se basó en 15 transcripciones de entrevistas a profesionales. Véase la sección 3.2, “Thematic Analysis”.

Resumen personal

Básicamente Schwarz y sus coautores investigaron qué pasa cuando cada equipo de trabajo usa las tecnologías que quiere en microservicios. Entrevistaron a profesionales para ver qué funciona y qué sale mal al integrar servicios de otros equipos. Llegaron a la conclusión de que a veces cambiar de tecnología es por una buena razón del proyecto, pero otras veces no hay un motivo claro. Al final te sugieren 13 reglas para organizarse bien, como definir qué cosas sí o sí se deben estandarizar (ejemplo: los formatos para pasarse la información) y en qué cosas sí le dan libertad a cada equipo.

Nota personal y uso en TA-IND-01

Lo que entendí aquí es que usar tecnologías distintas está bien si cada una resuelve un problema específico, pero al final del día te puede complicar la vida a la hora de juntar todo. Esta fuente me sirve para la parte de arquitectura, para explicar qué tecnologías comparten los servicios y cuáles decide usar cada equipo por su cuenta.

Ficha 2. Seguridad de las API

Referencia IEEE

[2] P. Genfer, S. Serbout, G. Simhandl, U. Zdun y C. Pautasso, «Understanding security tactics in microservice APIs using annotated software architecture decomposition models – a controlled experiment,» *Empirical Software Engineering*, vol. 30, n.º 3, 2025, art. 66, doi: 10.1007/s10664-024-10601-1

Identificador verificado

DOI 10.1007/s10664-024-10601-1

Dato concreto y ubicación

El experimento se ejecutó en cuatro rondas con 70 participantes. Véase la sección 4.2.3, “Participants”.

Resumen personal

Genfer y sus coautores hicieron un experimento con personas para ver si poner notas de seguridad en los diagramas de componentes ayudaba a entender mejor las API. Pusieron a la gente a responder preguntas sobre dos sistemas (una vez con diagramas anotados y otra sin esas

anotaciones) y midieron qué tan bien respondían y cuánto tardaban. Resulta que las notas ayudaron muchísimo a dar respuestas correctas, sobre todo a los que tenían menos experiencia, aunque en el tiempo que tardaron no cambió mucho la cosa. Con esto demuestran que documentar bien la arquitectura sirve para entender los controles de seguridad.

Nota personal y uso en TA-IND-01

Me quedó claro que hacer un diagrama y ponerle notas de seguridad te ayuda para saber dónde están los controles. Pienso usar esto en la sección de seguridad para mostrar, por ejemplo, en qué parte exacta del sistema se valida quién es el que está mandando una solicitud.

Ficha 3. Rendimiento y escalabilidad

Referencia IEEE

[3] G. Blinowski, A. Ojdowska y A. Przybyłek, «Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,» *IEEE Access*, vol. 10, págs. 20 357-20 374, 2022, doi: 10.1109/ACCESS.2022.3152803

Identificador verificado

DOI 10.1109/ACCESS.2022.3152803. Versión publicada en el repositorio institucional.

Dato concreto y ubicación

En los experimentos sobre una sola máquina, la versión monolítica obtuvo mayor rendimiento que la versión de microservicios. Véase la sección VIII, p. 20372.

Resumen personal

Blinowski y sus coautores querían ver qué era más rápido y escalable: si una app tradicional (monolito) o una dividida en microservicios. Para probarlo, hicieron pruebas de estrés en Java y C# tanto en una computadora local como en la nube, específicamente en Azure App Service y en Azure Spring Cloud. Lo curioso fue que, en una sola máquina, la app monolítica funcionó mejor que la de microservicios. También se dieron cuenta de que meterle más servidores a lo loco no siempre ayuda y a veces hasta frena el sistema. Todo depende de la carga real que reciba el proyecto.

Nota personal y uso en TA-IND-01

Lo que aprendí aquí es que por más que dividas tu app en microservicios, no te asegura mágicamente que vaya a ser más rápida. En la parte de rendimiento usaría esta fuente para analizar si realmente vale la pena esa separación según la carga que reciba el sistema y cómo se despliegue.

Ficha 4. Patrones de diseño

Referencia IEEE

[4] B. Burns, *Designing Distributed Systems*, 2.ª ed. O'Reilly Media, 2024, ISBN: 9781098156343

Identificador verificado

ISBN 9781098156343 (edición digital).

Dato concreto y ubicación

El capítulo 3, "The Sidecar Pattern", presenta un ejemplo que añade HTTPS a un servicio heredado mediante un sidecar.

Resumen personal

Burns te enseña un truco para agregarle funciones a una aplicación sin tener que meterle mano al contenedor o código principal, usando algo llamado el patrón sidecar. La idea es poner un contenedor "ayudante" pegado al contenedor principal en la misma máquina para que compartan recursos (como la red). El ejemplo que da es un sistema viejo que solo acepta tráfico HTTP sin cifrar, al que le pegas este ayudante al lado para que él se encargue de gestionar la seguridad con HTTPS. Así te queda claro qué hace el servicio principal y de qué se encarga el contenedor auxiliar.

Nota personal y uso en TA-IND-01

Entendí que puedes meter un contenedor auxiliar que le haga el favor al servicio principal de encargarse de cosas como el HTTPS sin tocar el código original. Esto lo pondría en la sección de despliegue para explicar cómo se dividen el trabajo el servicio principal y sus ayudantes, si es que llego a usar este patrón en mi proyecto.

Ficha 5. Replicación de datos

Referencia IEEE

[5] M. Kleppmann y C. Riccomini, *Designing Data-Intensive Applications, The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, 2.ª ed. O'Reilly Media, 2026, ISBN: 9781098119058

Identificador verificado

ISBN 9781098119058 (edición digital).

Dato concreto y ubicación

El capítulo 6, "Replication", distingue tres enfoques: líder único, múltiples líderes y replicación sin líder.

Resumen personal

Kleppmann y Riccomini explican el dolor de cabeza que es mantener los mismos datos copiados en varias computadoras mientras la gente los va cambiando. Comparan tres formas de hacerlo: cuando hay una sola máquina líder, cuando hay varias líderes o cuando no hay ninguna. Dan tres razones de por qué vale la pena complicarse con esto: para que la app responda más rápido si el usuario está cerca del servidor, para que todo siga funcionando si se cae una máquina, y para repartir la carga de trabajo entre varias compus. Eso sí, asumen que cada máquina guardará una copia completa.

Nota personal y uso en TA-IND-01

Aquí entendí que tener varias copias de los datos te salvan si algo falla, pero algo a tener en cuenta, esas copias a veces se demoran un poco en actualizarse. Usaría esta fuente en la sección de manejo de datos para explicar cómo viajan las escrituras y qué pasa si una réplica se queda atascada.

Declaración de uso de inteligencia artificial

Se utilizó Codex de OpenAI para localizar fuentes, preparar ejemplos de apoyo.